

# Unsupervised Variance Maximization, Reconstruction Error Minimization, and Eigen-Decomposition

A Unified Mathematical, Information-Theoretic, and Physical Foundation of  
Principal Component Analysis (PCA)

---

## Contents

|          |                                                                              |          |
|----------|------------------------------------------------------------------------------|----------|
| <b>1</b> | <b>The Unsupervised Linear Subspace Learning Paradigm</b>                    | <b>2</b> |
| <b>2</b> | <b>Dual Derivations of PCA</b>                                               | <b>2</b> |
| 2.1      | Derivation 1: Maximum Variance Formulation . . . . .                         | 2        |
| 2.2      | Derivation 2: Minimum Reconstruction Error Formulation . . . . .             | 3        |
| <b>3</b> | <b>Computation: SVD vs. Covariance Eigen-Decomposition</b>                   | <b>4</b> |
| 3.1      | Covariance Matrix Eigen-Decomposition . . . . .                              | 4        |
| 3.2      | Singular Value Decomposition (SVD) of Design Matrix . . . . .                | 4        |
| 3.3      | Mathematical Connection Between SVD and Eigen-Decomposition . . . . .        | 5        |
| <b>4</b> | <b>Information-Theoretic View and Physical Analogies</b>                     | <b>5</b> |
| 4.1      | Information-Theoretic View: Differential Entropy Maximization . . . . .      | 5        |
| 4.2      | Explained Variance Ratio (EVR) . . . . .                                     | 5        |
| 4.3      | Physical Analogy: Moment of Inertia and Principal Axes of Rotation . . . . . | 5        |
| <b>5</b> | <b>Kernel PCA (Non-Linear Subspace Generalization)</b>                       | <b>6</b> |
| 5.1      | Kernelized Covariance in RKHS . . . . .                                      | 6        |
| 5.2      | Double Centering of the Kernel Matrix . . . . .                              | 6        |
| <b>6</b> | <b>Production-Grade Vectorized NumPy Implementation</b>                      | <b>7</b> |

# 1 The Unsupervised Linear Subspace Learning Paradigm

High-dimensional data in  $\mathbb{R}^d$  frequently suffers from the **Curse of Dimensionality**: computational costs scale exponentially, sample density vanishes, and geometric distance metrics lose contrast. However, real-world data is rarely distributed uniformly throughout  $\mathbb{R}^d$ ; instead, it tends to concentrate along a lower-dimensional linear or non-linear manifold.

**Principal Component Analysis (PCA)**, originally formulated by Karl Pearson (1901) and independently expanded by Harold Hotelling (1933), is the foundational unsupervised linear dimensionality reduction algorithm. PCA seeks an orthogonal linear transformation that maps vectors from high-dimensional space  $\mathbb{R}^d$  into a lower-dimensional subspace  $\mathbb{R}^k$  ( $k \ll d$ ) while preserving maximal statistical variance and structural information.

**Definition 1.1 (Unsupervised Feature Projection).** Let  $\mathcal{D} = \{\mathbf{x}^{(1)}, \mathbf{x}^{(2)}, \dots, \mathbf{x}^{(N)}\}$  be an unlabelled empirical dataset of  $N$  observations sampled from  $p_{\text{data}}(\mathbf{x})$ , where  $\mathbf{x}^{(i)} \in \mathbb{R}^d$ .

The dataset is represented by the Design Matrix  $X \in \mathbb{R}^{N \times d}$ . We assume without loss of generality that  $X$  has been mean-centered:

$$\boldsymbol{\mu} = \frac{1}{N} \sum_{i=1}^N \mathbf{x}^{(i)} = \mathbf{0} \in \mathbb{R}^d \quad (1)$$

PCA seeks an orthonormal matrix  $\mathbf{V}_k = [\mathbf{v}_1, \mathbf{v}_2, \dots, \mathbf{v}_k] \in \mathbb{R}^{d \times k}$  (where  $\mathbf{V}_k^T \mathbf{V}_k = \mathbf{I}_k$ ) that projects centered data vectors  $\mathbf{x}^{(i)} \in \mathbb{R}^d$  to lower-dimensional latent coordinates  $\mathbf{z}^{(i)} \in \mathbb{R}^k$ :

$$\mathbf{z}^{(i)} = \mathbf{V}_k^T \mathbf{x}^{(i)} \iff Z = X \mathbf{V}_k \in \mathbb{R}^{N \times k} \quad (2)$$

## 2 Dual Derivations of PCA

PCA can be rigorously derived from two completely distinct geometric perspectives: 1. **Maximum Variance Projection**: Finding directions that maximize the spread (variance) of projected data. 2. **Minimum Reconstruction Error**: Finding an orthogonal linear subspace that minimizes the  $L_2$  Euclidean distance between original vectors and their low-rank reconstructions.

We prove below that both formulations lead to the exact same linear subspace defined by the eigenvectors of the sample covariance matrix.

### 2.1 Derivation 1: Maximum Variance Formulation

Consider projecting  $N$  mean-centered vectors  $\mathbf{x}^{(i)}$  onto a single unit vector  $\mathbf{v}_1 \in \mathbb{R}^d$  ( $\|\mathbf{v}_1\|_2^2 = \mathbf{v}_1^T \mathbf{v}_1 = 1$ ).

The scalar projection of  $\mathbf{x}^{(i)}$  onto  $\mathbf{v}_1$  is  $z_1^{(i)} = \mathbf{v}_1^T \mathbf{x}^{(i)}$ . The sample mean of the projected coordinates is zero:

$$\bar{z}_1 = \frac{1}{N} \sum_{i=1}^N \mathbf{v}_1^T \mathbf{x}^{(i)} = \mathbf{v}_1^T \left( \frac{1}{N} \sum_{i=1}^N \mathbf{x}^{(i)} \right) = 0 \quad (3)$$

The empirical variance of the projected data along  $\mathbf{v}_1$  is:

$$\begin{aligned} \text{Var}(z_1) &= \frac{1}{N-1} \sum_{i=1}^N (z_1^{(i)} - \bar{z}_1)^2 = \frac{1}{N-1} \sum_{i=1}^N (\mathbf{v}_1^T \mathbf{x}^{(i)})^2 \\ &= \frac{1}{N-1} \sum_{i=1}^N \mathbf{v}_1^T \mathbf{x}^{(i)} \mathbf{x}^{(i)T} \mathbf{v}_1 = \mathbf{v}_1^T \left( \frac{1}{N-1} \sum_{i=1}^N \mathbf{x}^{(i)} \mathbf{x}^{(i)T} \right) \mathbf{v}_1 \\ &= \mathbf{v}_1^T \boldsymbol{\Sigma} \mathbf{v}_1 \end{aligned} \quad (4)$$

where  $\boldsymbol{\Sigma} = \frac{1}{N-1} X^T X \in \mathbb{R}^{d \times d}$  is the unbiased sample covariance matrix.

**Definition 2.1 (Maximum Variance Primal Optimization).** To find the first principal direction  $\mathbf{v}_1$ , we maximize projected variance subject to the unit-norm constraint:

$$\max_{\mathbf{v}_1} \mathbf{v}_1^T \mathbf{\Sigma} \mathbf{v}_1 \quad \text{subject to} \quad \mathbf{v}_1^T \mathbf{v}_1 = 1 \quad (5)$$

We formulate the Lagrangian  $\mathcal{L}(\mathbf{v}_1, \lambda_1)$  with multiplier  $\lambda_1$ :

$$\mathcal{L}(\mathbf{v}_1, \lambda_1) = \mathbf{v}_1^T \mathbf{\Sigma} \mathbf{v}_1 - \lambda_1 (\mathbf{v}_1^T \mathbf{v}_1 - 1) \quad (6)$$

Setting the gradient with respect to  $\mathbf{v}_1$  to zero:

$$\nabla_{\mathbf{v}_1} \mathcal{L} = 2\mathbf{\Sigma} \mathbf{v}_1 - 2\lambda_1 \mathbf{v}_1 = \mathbf{0} \implies \mathbf{\Sigma} \mathbf{v}_1 = \lambda_1 \mathbf{v}_1 \quad (7)$$

This reveals that  $\mathbf{v}_1$  must be an **\*\*eigenvector\*\*** of the sample covariance matrix  $\mathbf{\Sigma}$ , and  $\lambda_1$  is its corresponding eigenvalue.

Left-multiplying by  $\mathbf{v}_1^T$ :

$$\mathbf{v}_1^T \mathbf{\Sigma} \mathbf{v}_1 = \lambda_1 \mathbf{v}_1^T \mathbf{v}_1 = \lambda_1 \quad (8)$$

Thus, the projected variance equals the eigenvalue  $\lambda_1$ . To maximize variance,  $\mathbf{v}_1$  must be chosen as the eigenvector corresponding to the **\*\*largest eigenvalue\*\***  $\lambda_1 = \lambda_{\max}(\mathbf{\Sigma})$ .

Subsequent principal components  $\mathbf{v}_k$  are found iteratively by maximizing variance subject to unit-norm and orthogonality constraints to all preceding components  $\mathbf{v}_1, \dots, \mathbf{v}_{k-1}$ :

$$\mathbf{\Sigma} \mathbf{v}_k = \lambda_k \mathbf{v}_k, \quad \text{with} \quad \mathbf{v}_k^T \mathbf{v}_l = \delta_{kl} \quad (9)$$

## 2.2 Derivation 2: Minimum Reconstruction Error Formulation

Instead of maximizing variance, consider approximating a  $d$ -dimensional vector  $\mathbf{x}^{(i)}$  using a linear combination of  $k < d$  orthonormal basis vectors  $\mathbf{v}_1, \dots, \mathbf{v}_k$ :

$$\hat{\mathbf{x}}^{(i)} = \sum_{j=1}^k z_j^{(i)} \mathbf{v}_j = \sum_{j=1}^k \left( \mathbf{x}^{(i)T} \mathbf{v}_j \right) \mathbf{v}_j = \mathbf{V}_k \mathbf{V}_k^T \mathbf{x}^{(i)} \quad (10)$$

where  $\mathbf{V}_k \mathbf{V}_k^T \in \mathbb{R}^{d \times d}$  is the orthogonal projection matrix onto the  $k$ -dimensional subspace.

The reconstruction error vector is  $\mathbf{e}^{(i)} = \mathbf{x}^{(i)} - \hat{\mathbf{x}}^{(i)}$ . Extending  $\mathbf{V}_k$  to a complete orthonormal basis of  $\mathbb{R}^d$  with  $[\mathbf{V}_k, \mathbf{V}_{d-k}]$ :

$$\mathbf{x}^{(i)} = \sum_{j=1}^d \left( \mathbf{x}^{(i)T} \mathbf{v}_j \right) \mathbf{v}_j \implies \mathbf{e}^{(i)} = \sum_{j=k+1}^d \left( \mathbf{x}^{(i)T} \mathbf{v}_j \right) \mathbf{v}_j \quad (11)$$

Evaluating total Mean Squared Reconstruction Error (MSRE) across all  $N$  samples:

$$\begin{aligned} J_{\text{recon}}(\mathbf{V}_d) &= \frac{1}{N-1} \sum_{i=1}^N \left\| \mathbf{x}^{(i)} - \hat{\mathbf{x}}^{(i)} \right\|_2^2 = \frac{1}{N-1} \sum_{i=1}^N \left\| \sum_{j=k+1}^d \left( \mathbf{x}^{(i)T} \mathbf{v}_j \right) \mathbf{v}_j \right\|_2^2 \\ &= \frac{1}{N-1} \sum_{i=1}^N \sum_{j=k+1}^d \left( \mathbf{v}_j^T \mathbf{x}^{(i)} \mathbf{x}^{(i)T} \mathbf{v}_j \right) \\ &= \sum_{j=k+1}^d \mathbf{v}_j^T \left( \frac{1}{N-1} \sum_{i=1}^N \mathbf{x}^{(i)} \mathbf{x}^{(i)T} \right) \mathbf{v}_j = \sum_{j=k+1}^d \mathbf{v}_j^T \mathbf{\Sigma} \mathbf{v}_j \end{aligned} \quad (12)$$

Under eigenvector basis  $\mathbf{\Sigma} \mathbf{v}_j = \lambda_j \mathbf{v}_j$ :

$$J_{\text{recon}} = \sum_{j=k+1}^d \lambda_j \quad (13)$$

To minimize the sum of discarded eigenvalues  $\sum_{j=k+1}^d \lambda_j$ , we must discard the  $d - k$  smallest eigenvalues of  $\mathbf{\Sigma}$ . Consequently, the optimal projection subspace  $\mathbf{V}_k$  must be spanned by the eigenvectors corresponding to the **\*\* $k$  largest eigenvalues\*\***.

**Theorem 2.1 (Duality Equivalence Theorem).** Maximizing projected variance and minimizing reconstruction error are mathematically dual problems that produce identical principal subspaces:

$$\underbrace{\sum_{j=1}^d \lambda_j}_{\text{Total Variance}} = \underbrace{\sum_{j=1}^k \lambda_j}_{\text{Retained Projected Variance}} + \underbrace{\sum_{j=k+1}^d \lambda_j}_{\text{Reconstruction Error}} \quad (14)$$

### Geometric Geometry of PCA Subspace Projection

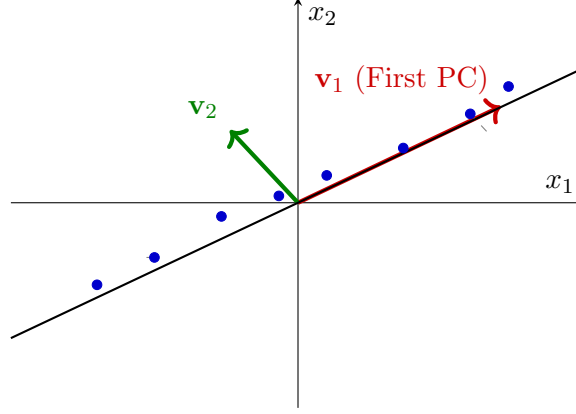


Figure 1: Principal Component Analysis on 2D correlated data.  $\mathbf{v}_1$  lies along the axis of maximum variance (and minimum orthogonal projection distance), while  $\mathbf{v}_2$  captures residual variance.

## 3 Computation: SVD vs. Covariance Eigen-Decomposition

Two numerical approaches exist for computing principal components: direct **Covariance Eigen-Decomposition** and **Singular Value Decomposition (SVD)** of the design matrix.

### 3.1 Covariance Matrix Eigen-Decomposition

Because  $\Sigma = \frac{1}{N-1} X^T X \in \mathbb{R}^{d \times d}$  is symmetric and positive semi-definite, its Spectral Decomposition is:

$$\Sigma = \mathbf{V} \mathbf{\Lambda} \mathbf{V}^T \quad (15)$$

where  $\mathbf{V} = [\mathbf{v}_1, \dots, \mathbf{v}_d] \in \mathbb{R}^{d \times d}$  is an orthogonal matrix of eigenvectors, and  $\mathbf{\Lambda} = \text{diag}(\lambda_1, \dots, \lambda_d)$  contains eigenvalues in descending order  $\lambda_1 \geq \lambda_2 \geq \dots \geq \lambda_d \geq 0$ .

### 3.2 Singular Value Decomposition (SVD) of Design Matrix

Explicitly computing  $X^T X$  introduces numerical conditioning loss (squaring matrix condition numbers:  $\kappa(X^T X) = \kappa(X)^2$ ). Production-grade implementations evaluate PCA directly via SVD of the mean-centered design matrix  $X \in \mathbb{R}^{N \times d}$ :

$$X = \mathbf{U} \mathbf{S} \mathbf{V}^T \quad (16)$$

where:

- $\mathbf{U} \in \mathbb{R}^{N \times N}$  is an orthogonal matrix of left singular vectors (eigenvectors of  $X X^T$ ).
- $\mathbf{S} \in \mathbb{R}^{N \times d}$  is a diagonal matrix containing non-negative singular values  $s_1 \geq s_2 \geq \dots \geq s_{\min(N,d)} \geq 0$ .
- $\mathbf{V} \in \mathbb{R}^{d \times d}$  is an orthogonal matrix of right singular vectors (eigenvectors of  $X^T X$ ).

### 3.3 Mathematical Connection Between SVD and Eigen-Decomposition

Computing the sample covariance matrix using SVD expansion:

$$\begin{aligned}\Sigma &= \frac{1}{N-1} X^T X = \frac{1}{N-1} (\mathbf{U} \mathbf{S} \mathbf{V}^T)^T (\mathbf{U} \mathbf{S} \mathbf{V}^T) \\ &= \frac{1}{N-1} \mathbf{V} \mathbf{S}^T \underbrace{\mathbf{U}^T \mathbf{U}}_{\mathbf{I}_N} \mathbf{S} \mathbf{V}^T = \mathbf{V} \left( \frac{\mathbf{S}^T \mathbf{S}}{N-1} \right) \mathbf{V}^T\end{aligned}\quad (17)$$

Comparing this directly with spectral decomposition  $\Sigma = \mathbf{V} \Lambda \mathbf{V}^T$ :

$$\Lambda = \frac{\mathbf{S}^2}{N-1} \implies \lambda_j = \frac{s_j^2}{N-1} \quad (18)$$

The right singular vectors  $\mathbf{V}$  of design matrix  $X$  are **identical** to the eigenvectors of covariance matrix  $\Sigma$ , and the eigenvalues  $\lambda_j$  are simple quadratic transformations of singular values  $s_j$ .

Transformed principal coordinates  $Z = X \mathbf{V}_k$  express directly via left singular vectors:

$$Z = (\mathbf{U} \mathbf{S} \mathbf{V}^T) \mathbf{V}_k = \mathbf{U}_k \mathbf{S}_k \quad (19)$$

## 4 Information-Theoretic View and Physical Analogies

### 4.1 Information-Theoretic View: Differential Entropy Maximization

Under continuous multivariate Gaussian assumptions  $\mathbf{X} \sim \mathcal{N}(\mathbf{0}, \Sigma)$ , the **Differential Entropy**  $h(\mathbf{X})$  of the vector distribution is:

$$h(\mathbf{X}) = - \int p(\mathbf{x}) \ln p(\mathbf{x}) d\mathbf{x} = \frac{1}{2} \ln \left( (2\pi e)^d |\Sigma| \right) = \frac{d}{2} (1 + \ln 2\pi) + \frac{1}{2} \sum_{j=1}^d \ln \lambda_j \quad (20)$$

Projecting  $\mathbf{X}$  onto a  $k$ -dimensional orthogonal subspace  $\mathbf{V}_k$  maximizes the joint differential entropy of the compressed representations:

$$h(\mathbf{Z}) = \frac{k}{2} (1 + \ln 2\pi) + \frac{1}{2} \sum_{j=1}^k \ln \lambda_j \quad (21)$$

Maximizing retained entropy is equivalent to maximizing the sum of log-eigenvalues  $\sum_{j=1}^k \ln \lambda_j$ , which requires choosing the  $k$  largest eigenvalues.

### 4.2 Explained Variance Ratio (EVR)

The proportion of total statistical variance retained by  $k$  components defines the **Explained Variance Ratio (EVR)**:

$$\text{EVR}(k) = \frac{\sum_{j=1}^k \lambda_j}{\sum_{m=1}^d \lambda_m} = \frac{\sum_{j=1}^k s_j^2}{\sum_{m=1}^d s_m^2} \in [0, 1] \quad (22)$$

### 4.3 Physical Analogy: Moment of Inertia and Principal Axes of Rotation

PCA mathematics maps onto the classical mechanics of rotating rigid bodies.

Consider a 3D physical cloud of particles with masses  $m_i = 1$  and position vectors  $\mathbf{r}^{(i)} = [x_1^{(i)}, x_2^{(i)}, x_3^{(i)}]^T$  relative to their center of mass.

The mechanical **Moment of Inertia Tensor**  $\mathbf{I} \in \mathbb{R}^{3 \times 3}$  is defined as:

$$\mathbf{I} = \sum_{i=1}^N \left( \|\mathbf{r}^{(i)}\|_2^2 \mathbf{I}_3 - \mathbf{r}^{(i)} \mathbf{r}^{(i)T} \right) = \text{Tr}(X^T X) \mathbf{I}_3 - X^T X \quad (23)$$

- The eigenvectors  $\mathbf{v}_j$  of  $X^T X$  (and thus  $\mathbf{\Sigma}$ ) correspond to the **\*\*Principal Axes of Inertia\*\*** of the physical body.
- Rotating a rigid body around its primary principal axis  $\mathbf{v}_1$  minimizes rotational kinetic wobble because it maximizes the body's principal moment of inertia.
- Principal components align with the natural rotational axes of mass symmetry, decoupling rotational angular momentum along orthogonal directions.

## 5 Kernel PCA (Non-Linear Subspace Generalization)

When data forms non-linear manifolds (e.g., concentric spheres or Swiss rolls), linear PCA projections overlap distinct clusters. Bernhard Schölkopf, Alexander Smola, and Klaus-Robert Müller (1998) introduced **\*\*Kernel PCA\*\*** to construct non-linear principal components.

### 5.1 Kernelized Covariance in RKHS

Let  $\phi : \mathbb{R}^d \rightarrow \mathcal{H}$  map inputs into an infinite-dimensional Reproducing Kernel Hilbert Space (RKHS)  $\mathcal{H}$ . Assuming centered mapped data  $\sum_{i=1}^N \phi(\mathbf{x}^{(i)}) = \mathbf{0}$ , the covariance operator in  $\mathcal{H}$  is:

$$\mathbf{\Sigma}_{\mathcal{H}} = \frac{1}{N} \sum_{i=1}^N \phi(\mathbf{x}^{(i)}) \phi(\mathbf{x}^{(i)})^T \quad (24)$$

The eigenvalue equation in  $\mathcal{H}$  is:

$$\mathbf{\Sigma}_{\mathcal{H}} \mathbf{v} = \lambda \mathbf{v} \implies \left( \frac{1}{N} \sum_{i=1}^N \phi(\mathbf{x}^{(i)}) \phi(\mathbf{x}^{(i)})^T \right) \mathbf{v} = \lambda \mathbf{v} \quad (25)$$

Every eigenvector  $\mathbf{v} \in \mathcal{H}$  lies in the span of mapped points:

$$\mathbf{v} = \sum_{i=1}^N \alpha_i \phi(\mathbf{x}^{(i)}) \quad (26)$$

Substituting this expansion into the eigenvalue equation and multiplying by  $\phi(\mathbf{x}^{(k)})^T$  yields:

$$\frac{1}{N} \sum_{i=1}^N \alpha_i \phi(\mathbf{x}^{(k)})^T \sum_{j=1}^N \phi(\mathbf{x}^{(j)}) \phi(\mathbf{x}^{(j)})^T \phi(\mathbf{x}^{(i)}) = \lambda \sum_{i=1}^N \alpha_i \phi(\mathbf{x}^{(k)})^T \phi(\mathbf{x}^{(i)}) \quad (27)$$

Substituting the Kernel evaluation  $K_{ij} = K(\mathbf{x}^{(i)}, \mathbf{x}^{(j)}) = \langle \phi(\mathbf{x}^{(i)}), \phi(\mathbf{x}^{(j)}) \rangle_{\mathcal{H}}$ :

$$\frac{1}{N} \mathbf{K}^2 \boldsymbol{\alpha} = \lambda \mathbf{K} \boldsymbol{\alpha} \iff \mathbf{K} \boldsymbol{\alpha} = N \lambda \boldsymbol{\alpha} \quad (28)$$

This simplifies to an eigen-decomposition of the  $N \times N$  Gram matrix  $\mathbf{K}$ .

### 5.2 Double Centering of the Kernel Matrix

In general, mapped features  $\phi(\mathbf{x}^{(i)})$  are not zero-centered in  $\mathcal{H}$ . To center them without explicit mapping, we compute the **\*\*Gram Matrix Double Centering\*\***:

$$\mathbf{K}_C = \mathbf{H} \mathbf{K} \mathbf{H} \quad (29)$$

where  $\mathbf{H} = \mathbf{I}_N - \frac{1}{N} \mathbf{1}_N \mathbf{1}_N^T$  is the  $N \times N$  **\*\*Centering Matrix\*\***.

Solving  $\mathbf{K}_C \boldsymbol{\alpha}_k = \lambda_k^K \boldsymbol{\alpha}_k$  and normalizing  $\|\mathbf{v}_k\|_{\mathcal{H}}^2 = 1 \implies \lambda_k^K \boldsymbol{\alpha}_k^T \boldsymbol{\alpha}_k = 1$ , the projected non-linear coordinate  $z_k$  for a test point  $\mathbf{x}^*$  is:

$$z_k(\mathbf{x}^*) = \mathbf{v}_k^T \phi(\mathbf{x}^*) = \sum_{i=1}^N \alpha_{i,k} K(\mathbf{x}^{(i)}, \mathbf{x}^*) \quad (30)$$

## 6 Production-Grade Vectorized NumPy Implementation

The following Python class implements a unified, production-grade PCA and Kernel PCA Engine using SVD and Gram double-centering.

```
1 import numpy as np
2
3 class UniversalPCAEngine:
4     """
5     Production-grade Principal Component Analysis (PCA) Engine supporting:
6     1. Fast, numerically stable SVD-based Linear PCA.
7     2. Kernel PCA (RBF, Polynomial) via Gram Matrix Double-Centering.
8     3. Reconstruction, Explained Variance Ratios, and Subspace Projection.
9     """
10    def __init__(self, n_components: int = 2, kernel: str = 'linear',
11                 gamma: float = 0.1, degree: int = 3):
12        self.n_components = n_components
13        self.kernel = kernel
14        self.gamma = gamma
15        self.degree = degree
16
17        self.mean_ = None           # Feature means (d,)
18        self.components_ = None     # Principal directions V_k (k, d)
19        self.explained_variance_ = None # Eigenvalues lambda_j (k,)
20        self.explained_variance_ratio_ = None # EVR (k,)
21        self.singular_values_ = None # Singular values s_j (k,)
22
23        # Kernel PCA Attributes
24        self.X_fit_ = None          # Stored training points (N, d)
25        self.K_fit_rows_ = None     # Saved mean row sums for test
26
27        transform
28        self.K_fit_all_ = None      # Saved global mean scalar for
29        test transform
30        self.alphas_ = None         # Normalized Dual Eigenvectors (N,
31                                   k)
32
33    def _compute_kernel_matrix(self, X1: np.ndarray, X2: np.ndarray) -> np.
34    ndarray:
35        """Computes Kernel Gram Matrix K(X1, X2)."""
36        if self.kernel == 'rbf':
37            X1_sq = np.sum(X1**2, axis=1, keepdims=True)
38            X2_sq = np.sum(X2**2, axis=1, keepdims=True).T
39            dists_sq = np.maximum(X1_sq + X2_sq - 2.0 * (X1 @ X2.T), 0.0)
40            return np.exp(-self.gamma * dists_sq)
41        elif self.kernel == 'poly':
42            return (self.gamma * (X1 @ X2.T) + 1.0) ** self.degree
43        else:
44            return X1 @ X2.T
45
46    def fit(self, X: np.ndarray):
47        """Fits PCA model parameters on input dataset X (N, d)."""
48        X = np.asarray(X, dtype=np.float64)
49        N, d = X.shape
50
51        if self.kernel == 'linear':
52            # --- LINEAR SVD-BASED PCA ---
53            self.mean_ = np.mean(X, axis=0)
54            X_centered = X - self.mean_
55
56            # SVD of centered design matrix: X_centered = U * S * V^T
```

```

52         U, S, Vt = np.linalg.svd(X_centered, full_matrices=False)
53
54         # Eigenvalues:  $\lambda_j = s_j^2 / (N - 1)$ 
55         explained_var = (S**2) / (N - 1)
56         total_var = np.sum(explained_var)
57
58         # Store top k components
59         k = min(self.n_components, d)
60         self.components_ = Vt[:k, :] # (k, d)
61         self.explained_variance_ = explained_var[:k] # (k,)
62         self.explained_variance_ratio_ = explained_var[:k] / total_var
# (k,)
63         self.singular_values_ = S[:k] # (k,)
64
65     else:
66         # --- KERNEL PCA (DOUBLE-CENTERING) ---
67         self.X_fit_ = X.copy()
68         K = self._compute_kernel_matrix(X, X)
69
70         # Gram Double-Centering:  $K_C = H * K * H$ 
71         #  $H = I - (1/N) * 11^T$ 
72         self.K_fit_rows_ = np.mean(K, axis=1, keepdims=True) # (N, 1)
73         self.K_fit_all_ = np.mean(K) # scalar
74
75         K_C = K - self.K_fit_rows_ - self.K_fit_rows_.T + self.
K_fit_all_
76
77         # Spectral decomposition of symmetric centered Gram matrix
78         eigvals, eigvecs = np.linalg.eigh(K_C)
79
80         # Sort by descending eigenvalues
81         sorted_indices = np.argsort(eigvals)[::-1]
82         eigvals = eigvals[sorted_indices]
83         eigvecs = eigvecs[:, sorted_indices]
84
85         # Keep positive eigenvalues for top k
86         k = self.n_components
87         positive_mask = eigvals > 1e-10
88         k = min(k, np.sum(positive_mask))
89
90         self.explained_variance_ = eigvals[:k] / (N - 1)
91         self.explained_variance_ratio_ = eigvals[:k] / np.sum(eigvals[
positive_mask])
92
93         # Normalize dual eigenvectors:  $\|v_k\|_H = 1 \Rightarrow \alpha_k =$ 
eigvector /  $\sqrt{\lambda_k^K}$ 
94         self.alphas_ = eigvecs[:, :k] / np.sqrt(np.maximum(eigvals[:k],
1e-10))
95
96         return self
97
98     def transform(self, X: np.ndarray) -> np.ndarray:
99         """Projects input matrix X (N_query, d) onto principal component
subspace."""
100         X = np.asarray(X, dtype=np.float64)
101
102         if self.kernel == 'linear':
103             X_centered = X - self.mean_
104             return X_centered @ self.components_.T # (N_query, k)
105         else:

```



```

106         # Kernel PCA Transform
107         K_test = self._compute_kernel_matrix(X, self.X_fit_) # (N_query
, N_fit)
108
109         # Center test Kernel matrix using fit statistics
110         K_test_rows = np.mean(K_test, axis=1, keepdims=True)
111         K_C_test = K_test - K_test_rows - self.K_fit_rows_.T + self.
K_fit_all_
112
113         return K_C_test @ self.alphas_ # (N_query, k)
114
115     def inverse_transform(self, Z: np.ndarray) -> np.ndarray:
116         """Reconstructs original d-dimensional space vectors from low-rank
Z (N, k)."""
117         if self.kernel != 'linear':
118             raise NotImplementedError("Exact inverse transform is available
only for linear PCA.")
119
120         Z = np.asarray(Z, dtype=np.float64)
121         return (Z @ self.components_) + self.mean_
122
123
124 if __name__ == "__main__":
125     np.random.seed(42)
126
127     # 1. Validation Run: Linear PCA on 3D Gaussian Ellipsoid
128     N_samples = 500
129     cov_true = np.array([[3.0, 1.2, 0.2],
130                          [1.2, 1.0, 0.1],
131                          [0.2, 0.1, 0.2]])
132     X_3d = np.random.multivariate_normal([1.0, 2.0, -1.0], cov_true,
N_samples)
133
134     pca_linear = UniversalPCAEngine(n_components=2, kernel='linear')
135     pca_linear.fit(X_3d)
136     Z_2d = pca_linear.transform(X_3d)
137     X_reconstructed = pca_linear.inverse_transform(Z_2d)
138
139     reconstruction_mse = np.mean((X_3d - X_reconstructed)**2)
140
141     print("=== LINEAR SVD-PCA INITIALIZATION SUCCESSFUL ===")
142     print(f"Explained Variance Ratio (EVR) : {pca_linear.
explained_variance_ratio_}")
143     print(f"Cumulative Retained Variance : {np.sum(pca_linear.
explained_variance_ratio_)*100:.2f}%")
144     print(f"Reconstruction MSE (3D -> 2D) : {reconstruction_mse:.4f}\n")
145
146     # 2. Validation Run: Kernel PCA on Concentric Circles (Non-Linear)
147     r_in, r_out = np.random.uniform(0, 1, 150), np.random.uniform(2, 3,
150)
148     t_in, t_out = np.random.uniform(0, 2*np.pi, 150), np.random.uniform(0,
2*np.pi, 150)
149     X_circle = np.vstack([
150         np.column_stack([r_in * np.cos(t_in), r_in * np.sin(t_in)]),
151         np.column_stack([r_out * np.cos(t_out), r_out * np.sin(t_out)])
152     ])
153
154     kpca_rbf = UniversalPCAEngine(n_components=2, kernel='rbf', gamma=0.5)
155     kpca_rbf.fit(X_circle)
156     Z_kpca = kpca_rbf.transform(X_circle)

```

```
157
158     print("=== KERNEL PCA (RBF) INITIALIZATION SUCCESSFUL ===")
159     print(f"Original Non-Linear Shape : {X_circle.shape}")
160     print(f"Kernel Subspace Shape      : {Z_kpca.shape}")
```

Listing 1: Vectorized NumPy implementation of Linear PCA and Kernel PCA.